Exercicio 2 de Redes Neurais

Eduardo Rios de Andrade Sousa – 2022087105

1) Problema não linearmente separável

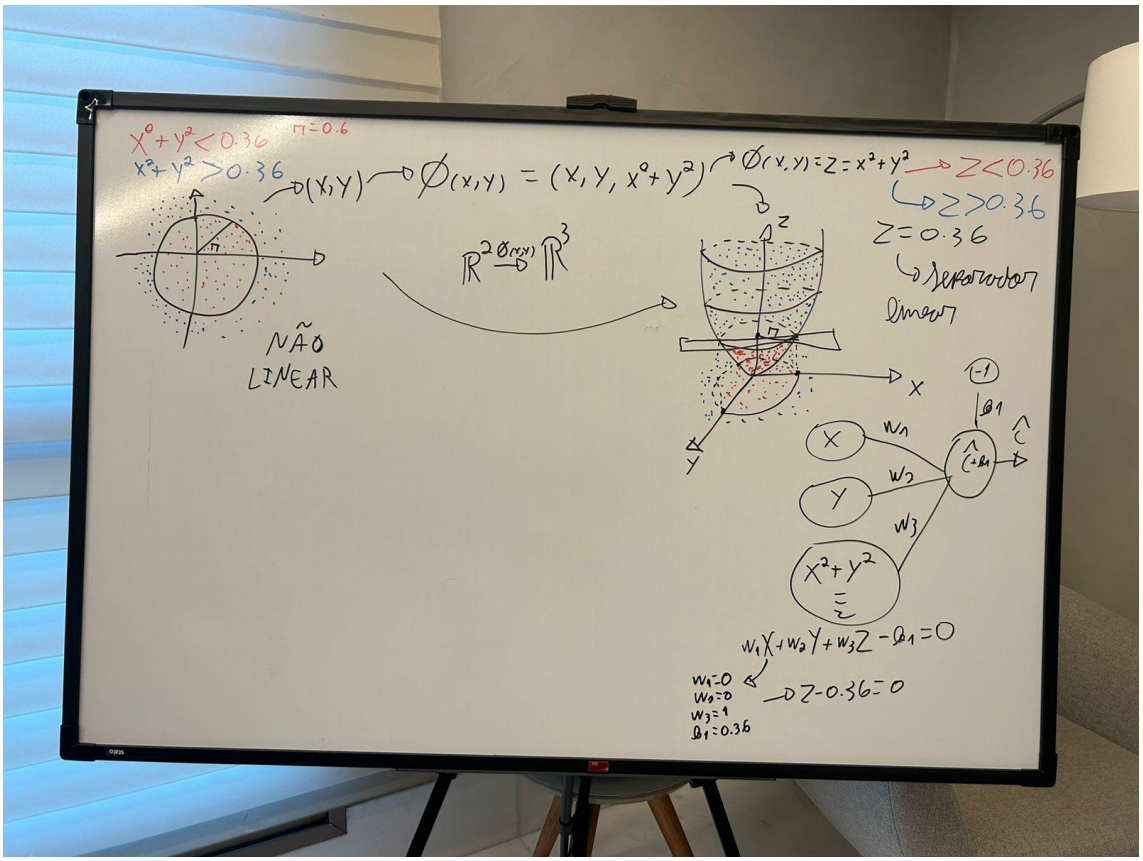
-Codigo Python

```
import numpy as np

import matplotlib.pyplot as plt


x = np.arange(-1, 1.1, 0.1) #x eh a matriz de entradas, gerada aleatoriamente entre -1 e 1

y= np.arange(-1, 1.1, 0.1) #y eh a matriz de entradas, gerada aleatoriamente entre -1 e 1


X, Y = np.meshgrid(x, y) #X e Y sao as matrizes de entradas, geradas a partir das matrizes x e y usando a funcao meshgrid


grid = np.column_stack((X.ravel(), Y.ravel())) #grid eh a matriz de entradas, gerada a partir das matrizes X e Y usando a funcao column_stack


plt.scatter(grid[:,0], grid[:,1], c='blue') #grafico dos pontos projetados neste novo espaço

plt.xlabel('X')

plt.ylabel('Y')

plt.title('Grafico dos pontos gerados originalmente sem separacao de classes')

plt.grid()

plt.show()


raio = 0.6
```

```
dist = np.sqrt(grid[:,0]**2 + grid[:,1]**2) #dist eh a matriz de distancias, calculada a partir das matrizes x e y usando a funcao sqrt

classe = (dist < raio)


plt.scatter(grid[classe,0], grid[classe,1], c='red') #grafico dos pontos projetados neste novo espaço, com as classes separadas

plt.scatter(grid[~classe,0], grid[~classe,1], c='blue') #grafico dos pontos projetados neste novo espaço, com as classes separadas

plt.xlabel('X')

plt.ylabel('Y')

plt.title('Grafico dos pontos gerados originalmente com separacao de classes')

plt.grid()

plt.show()


# Projecao nao linear usando a funcao de distancia ao centro, que resulta em uma superficie circular de separacao entre as classes


Z = dist**2 #Z eh a matriz de saida da camada oculta, gerada a partir da matriz grid usando a funcao de distancia ao centro


from mpl_toolkits.mplot3d import Axes3D


fig = plt.figure()

ax = fig.add_subplot(111, projection='3d')


ax.scatter(grid[classe,0], grid[classe,1], Z[classe], c='red')

ax.scatter(grid[~classe,0], grid[~classe,1], Z[~classe], c='blue')

r2 = raio**2


xx, yy = np.meshgrid(x, y)

zz = np.full_like(xx, r2)
```

```
ax.plot_surface(xx, yy, zz, alpha=0.3)


ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z = X² + Y²')


plt.title("Projecao nao linear dos dados")
plt.show()
```

- Calculos feitos



-Gráficos obtidos

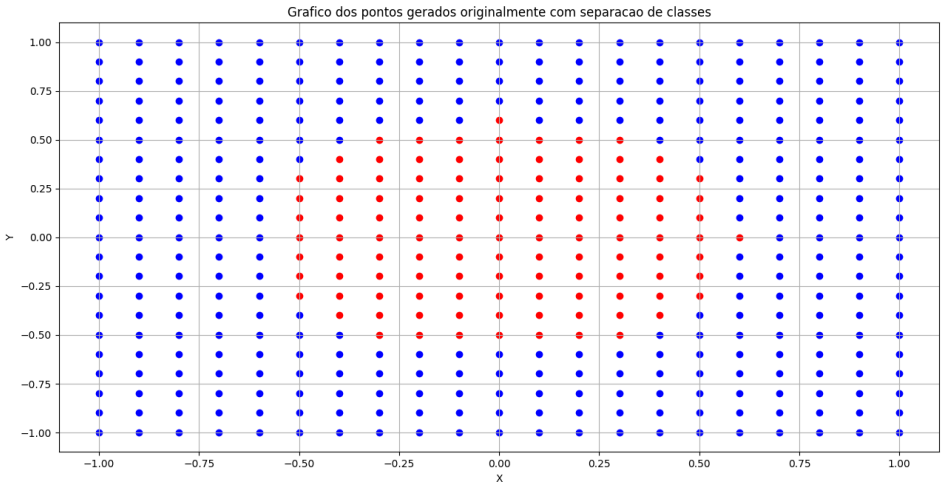
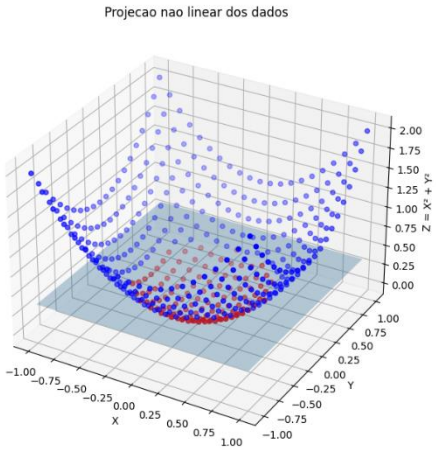
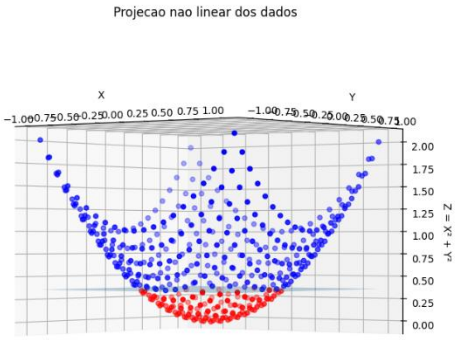


Figura 1.1 – Dados separados por classes de forma não linearmente separáveis





Figuras 1.2 – Mapeamento dos pontos para projeção vista a) de cima e b) da perspectiva de uma reta do plano separador

2) Overfitting e underfitting

- Sabendo-se que existe um ruido na amostragem, a curva em azul é a que melhor aproxima a função geradora dos dados. A curva vermelha está muito distante dos dados e gera um erro muito alto (underfitting). Já a curva de preto possui um erro de treinamento nulo, e se ajusta perfeitamente aos dados amostrados mesmo com ruídos (overfitting). Com isso, a curva em azul parece levar em consideração os padrões presentes nos dados e desconsidera os ruídos presentes nas amostras, fazendo com que seja o melhor modelo.

- O menor erro de treinamento é obtido pela curva em preto, pois ela passa exatamente por todos os pontos sem nenhum erro. Isso é clássico de overfitting, que é quando um modelo se ajusta demais aos dados de treinamento ao invés de aprender os padrões presentes nos dados para generalizar para um novo conjunto.

- Baixo erro de treinamento é, até certo ponto, um bom indicativo de que o modelo é bom. Quando o erro de treinamento passa a ser muito pequeno ou quase nulo, corremos o risco de causar overfitting no nosso modelo, o que fará com que ele não responda tão bem para os dados do conjunto de testes ou um conjunto de dados novos em atuação real. Devemos buscar um modelo que reduza o erro de treinamento sim, mas que seja capaz de apresentar erros pequenos também para novos conjuntos de dados, e isso só é possível se ao invés do modelo se ajustar exatamente aos dados de treino ele captar os padrões presentes em generalizar para novos dados da mesma natureza.

3) Aproximação polinomial

-Código python

```
import numpy as np

import matplotlib.pyplot as plt


def funcao_geradora(X):

   return 0.5*X**2 + 3*X + 10


X = np.random.uniform(-15, 10, 10) #X eh a matriz de entradas, gerada aleatoriamente entre -15 e 10

Y = funcao_geradora(X) + np.random.normal(0, 4, 10) #Y eh a matriz de saida, gerada a partir da funcao_geradora com adicao de ruido gaussiano


X_continuo = np.linspace(-15, 10, 100) #X_continuo eh a matriz de entradas para o grafico, gerada aleatoriamente entre -15 e 10

Y_geradora = funcao_geradora(X_continuo)


plt.plot(X, Y, 'o', label='Amostras', color='blue')

plt.xlabel('X')

plt.ylabel('Y')

plt.title('Amostras geradas a partir da função geradora + Funcao Geradora')

plt.plot(X_continuo, Y_geradora, '-', label='Funcao Geradora', color='green')

plt.legend()

plt.grid()

plt.show()


print(X.shape)

for p in range (1,9):

   H = np.array([[x**i for i in range(0,p+1)]for x in X])
```

```
    Hinv = np.linalg.pinv(H)
    W = np.dot(Hinv, Y)
    print("Para p = "+ str(p) +": "+ str(W))
    Yhat = np.dot(H, W)
    print("Polinomio obtido:")
    polinomio = "Y = "
    grau = len(W) - 1
    for i in range (grau, -1, -1):
        w = W[i]
        if i == grau:
            polinomio += f"{w:.4f}X^{i}"
        elif i != 0:
            polinomio += f" + {w:.4f}*X^{i}"
        else:
            polinomio += f" + {w:.4f}"
    print(polinomio)
    plt.plot(X, Y, 'o', label='Amostras', color='blue')
    plt.xlabel('X')
    plt.ylabel('Y')
    plt.title('Amostras geradas a partir da função geradora + Funcao Geradora')
    plt.plot(X_continuo, Y_geradora, '-', label='Funcao Geradora', color='green')
    plt.plot(X_continuo, np.dot(np.array([[x**i for i in range(0,p+1)]for x in
X_continuo]), W), '-', label='Polinomio de grau '+str(p), color='black')
    plt.legend()
    plt.grid()
    plt.show()


import numpy as np
import matplotlib.pyplot as plt
```

```
def funcao_geradora(X):
    return 0.5*X**2 + 3*X + 10


X = np.random.uniform(-15, 10, 100) #X eh a matriz de entradas, gerada aleatoriamente entre -15 e 10
Y = funcao_geradora(X) + np.random.normal(0, 4, 100) #Y eh a matriz de saida, gerada a partir da funcao_geradora com adicao de ruido gaussiano


X_continuo = np.linspace(-15, 10, 100) #X_continuo eh a matriz de entradas para o grafico, gerada aleatoriamente entre -15 e 10
Y_geradora = funcao_geradora(X_continuo)


plt.plot(X, Y, 'o', label='Amostras', color='blue')
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Amostras geradas a partir da função geradora + Funcao Geradora')
plt.plot(X_continuo, Y_geradora, '-', label='Funcao Geradora', color='green')
plt.legend()
plt.grid()
plt.show()


print(X.shape)
for p in range (1,9):
    H = np.array([[x**i for i in range(0,p+1)]for x in X])
    Hinv = np.linalg.pinv(H)
    W = np.dot(Hinv, Y)
    print("Para p = "+ str(p) +": "+ str(W))
    Yhat = np.dot(H, W)
    print("Polinomio obtido:")
```

```
polinomio = "Y = "
grau = len(W) - 1
for i in range (grau, -1, -1):
    w = W[i]
    if i == grau:
        polinomio += f"{w:.4f}X^{i}"
    elif i != 0:
        polinomio += f" + {w:.4f}*X^{i}"
    else:
        polinomio += f" + {w:.4f}"
print(polinomio)
plt.plot(X, Y, 'o', label='Amostras', color='blue')
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Amostras geradas a partir da função geradora + Funcao Geradora')
plt.plot(X_continuo, Y_geradora, '-', label='Funcao Geradora', color='green')
plt.plot(X_continuo, np.dot(np.array([[x**i for i in range(0,p+1)]for x in
X_continuo]), W), '-', label='Polinomio de grau '+ str(p), color='black')
plt.legend()
plt.grid()
plt.show()
```

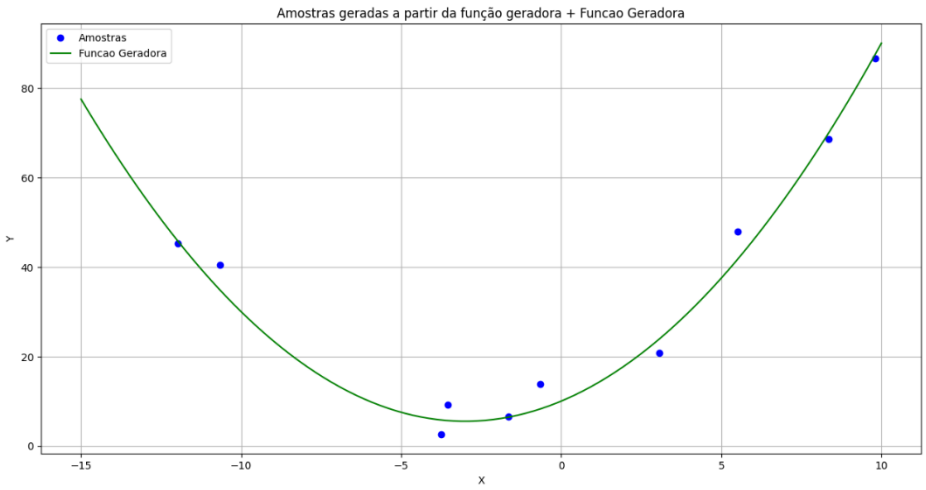
-Resultados obtidos

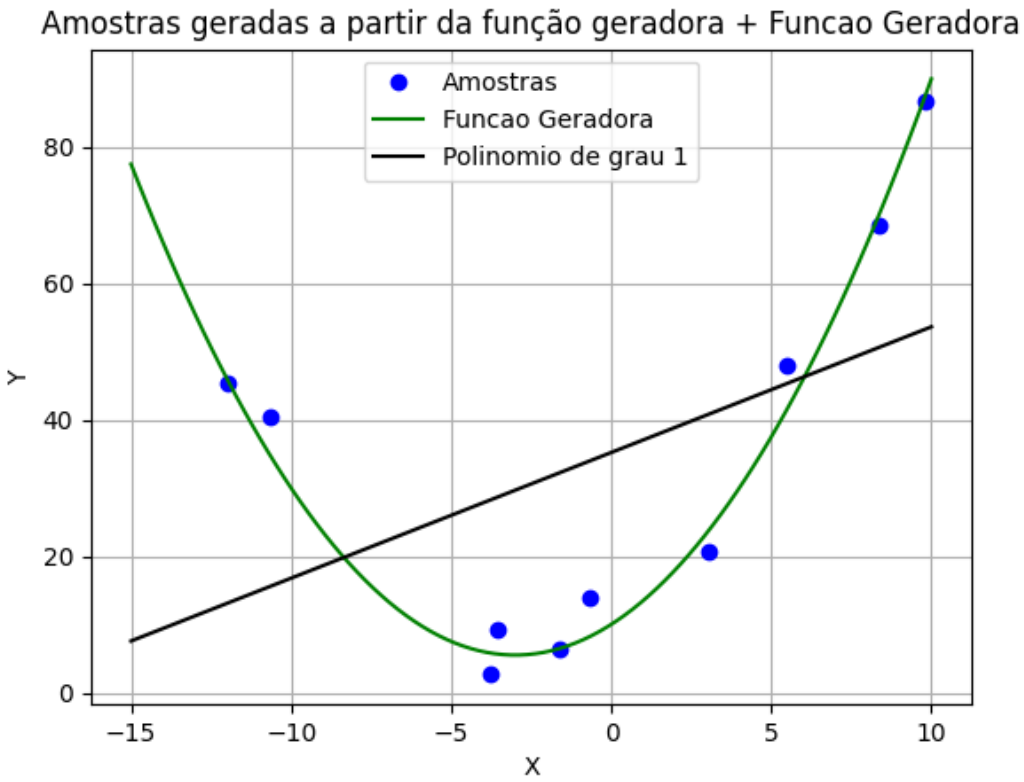
Para 10 pontos:

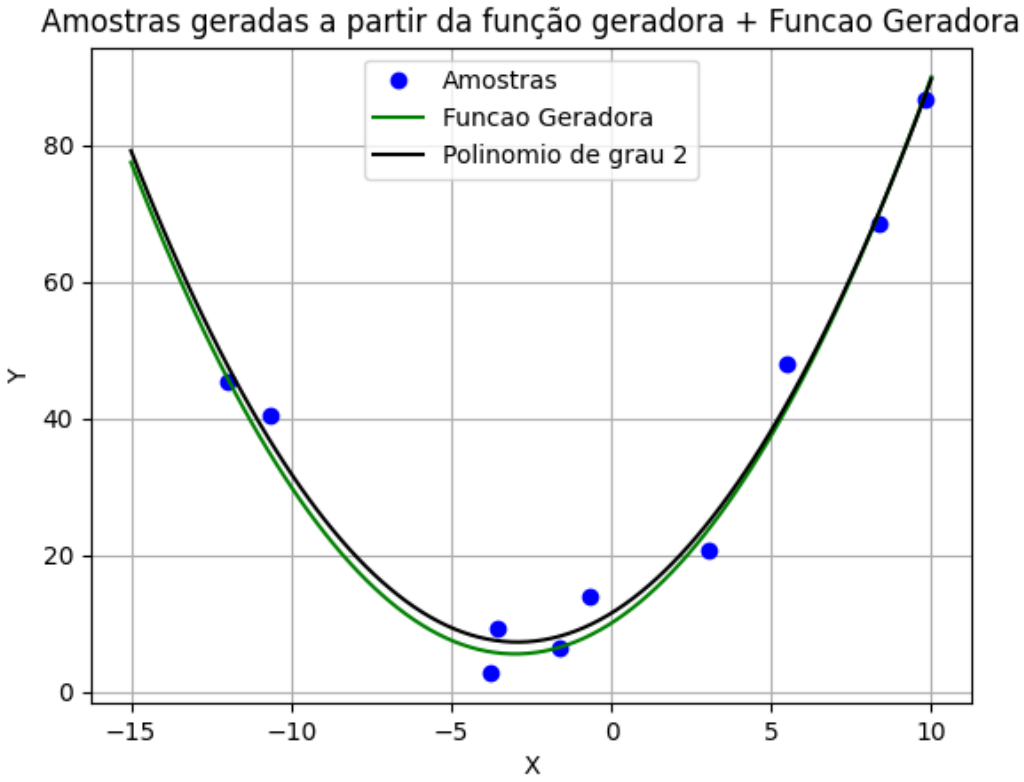
Polinômios obtidos:

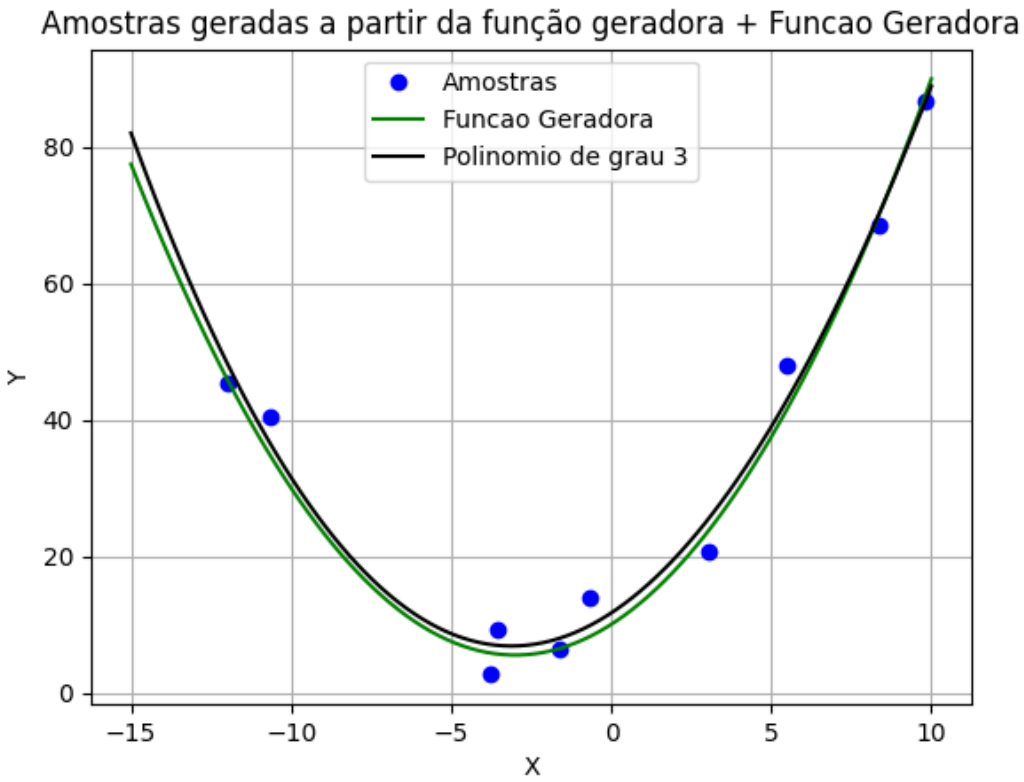
```
Para p = 1: [35.20939605  1.84319706]
Polinomio obtido:
Y = 1.8432X^1 + 35.2094
Para p = 2: [11.46201272  2.88964872  0.49369397]
Polinomio obtido:
Y = 0.4937X^2 + 2.8896*X^1 + 11.4620
Para p = 3: [ 1.16779368e+01  3.09056988e+00  4.85693270e-01 -2.20887011e-03]
Polinomio obtido:
Y = -0.0022X^3 + 0.4857*X^2 + 3.0906*X^1 + 11.6779
Para p = 4: [ 1.04134656e+01  3.06603918e+00  6.15075440e-01 -4.21940554e-03
 -1.09896034e-03]
Polinomio obtido:
Y = -0.0011X^4 + -0.0042*X^3 + 0.6151*X^2 + 3.0660*X^1 + 10.4135
Para p = 5: [ 1.10449303e+01  3.40810084e+00  5.67878173e-01 -1.81180665e-02
 -5.62266544e-04  1.03342409e-04]
Polinomio obtido:
Y = 0.0001X^5 + -0.0006*X^4 + -0.0181*X^3 + 0.5679*X^2 + 3.4081*X^1 + 11.0449
Para p = 6: [ 1.19880842e+01  3.23018927e+00  4.01063841e-01 -4.12827713e-03
  2.95908776e-03 -2.39342441e-05 -2.00678295e-05]
Polinomio obtido:
Y = -0.0000X^6 + -0.0000*X^5 + 0.0030*X^4 + -0.0041*X^3 + 0.4011*X^2 + 3.2302*X^1 + 11.9881
Para p = 7: [ 1.03598920e+01  1.32487616e-01  5.40926316e-01  2.69727044e-01
 -8.43082274e-03 -4.51632177e-03  7.92282148e-05  2.12471112e-05]
Polinomio obtido:
Y = 0.0000X^7 + 0.0001*X^6 + -0.0045*X^5 + -0.0084*X^4 + 0.2697*X^3 + 0.5409*X^2 + 0.1325*X^1 + 10.3599
Para p = 8: [ 1.45194433e+01  2.38748471e+00 -9.50395022e-01  6.46078844e-02
  9.29137499e-02 -3.55093307e-03 -1.39378685e-03  2.84245823e-05
  6.23228423e-06]
Polinomio obtido:
Y = 0.0000X^8 + 0.0000*X^7 + -0.0014*X^6 + -0.0036*X^5 + 0.0929*X^4 + 0.0646*X^3 + -0.9504*X^2 + 2.3875*X^1 + 14.5194
```

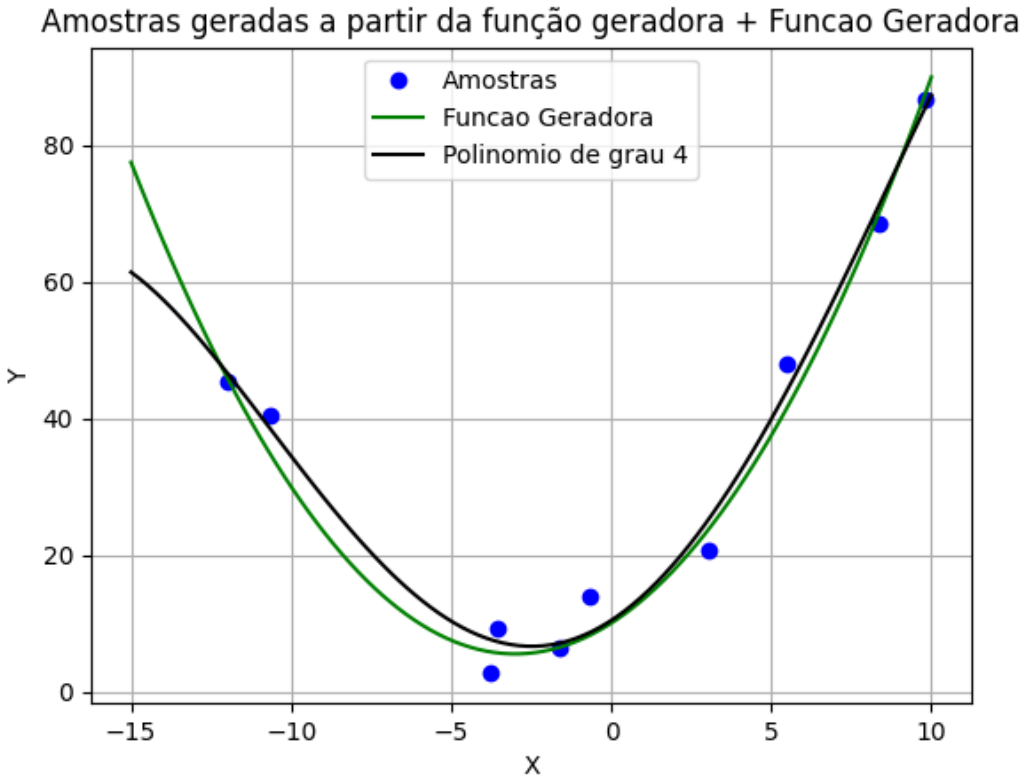
Gráficos:

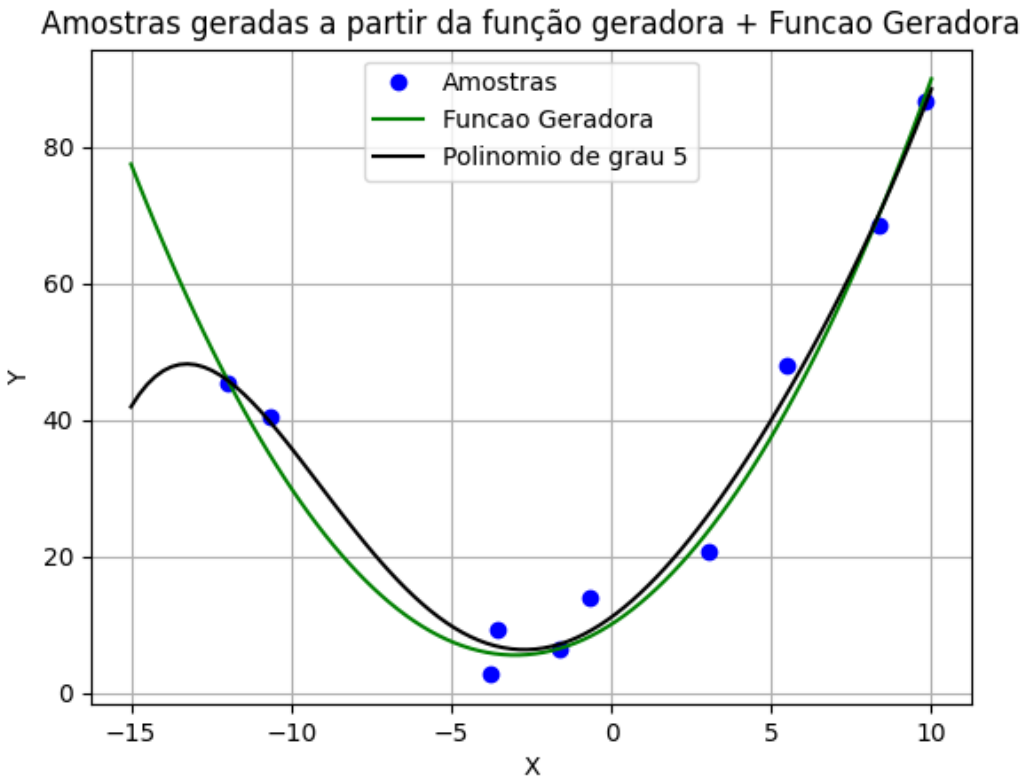


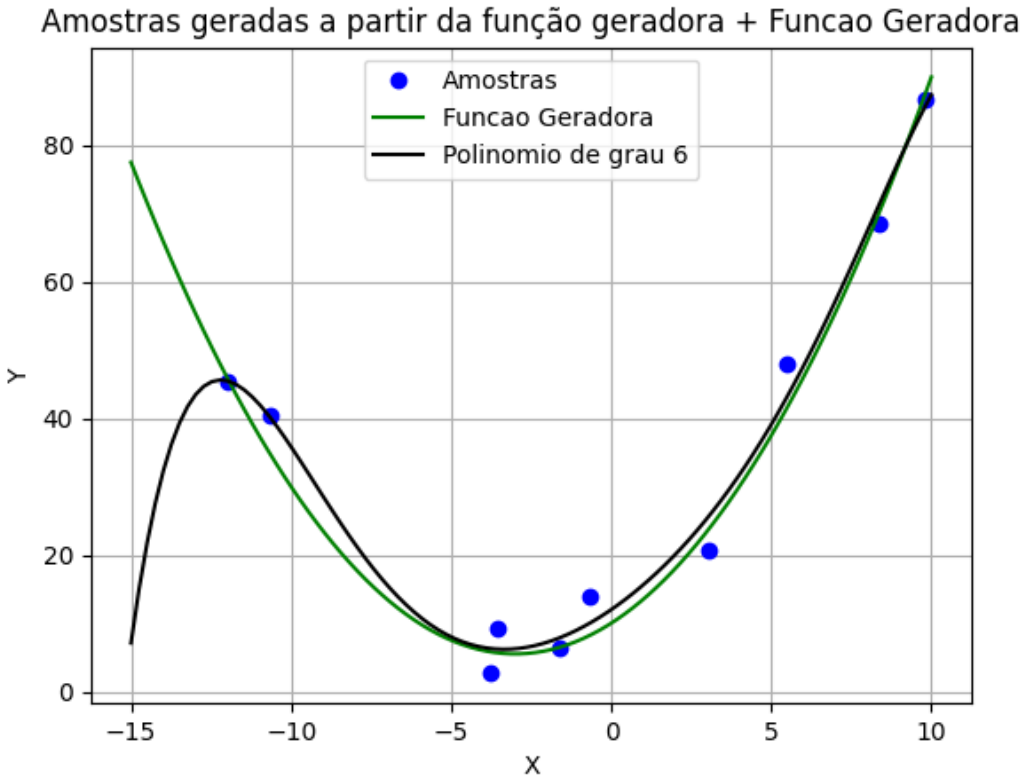

Amostras geradas a partir da função geradora + Funcao Geradora
Amostras
Funcao Geradora
Polinomio de grau 1
Y
X
0
20
40
60
80
−15
−10
−5
0
5
10

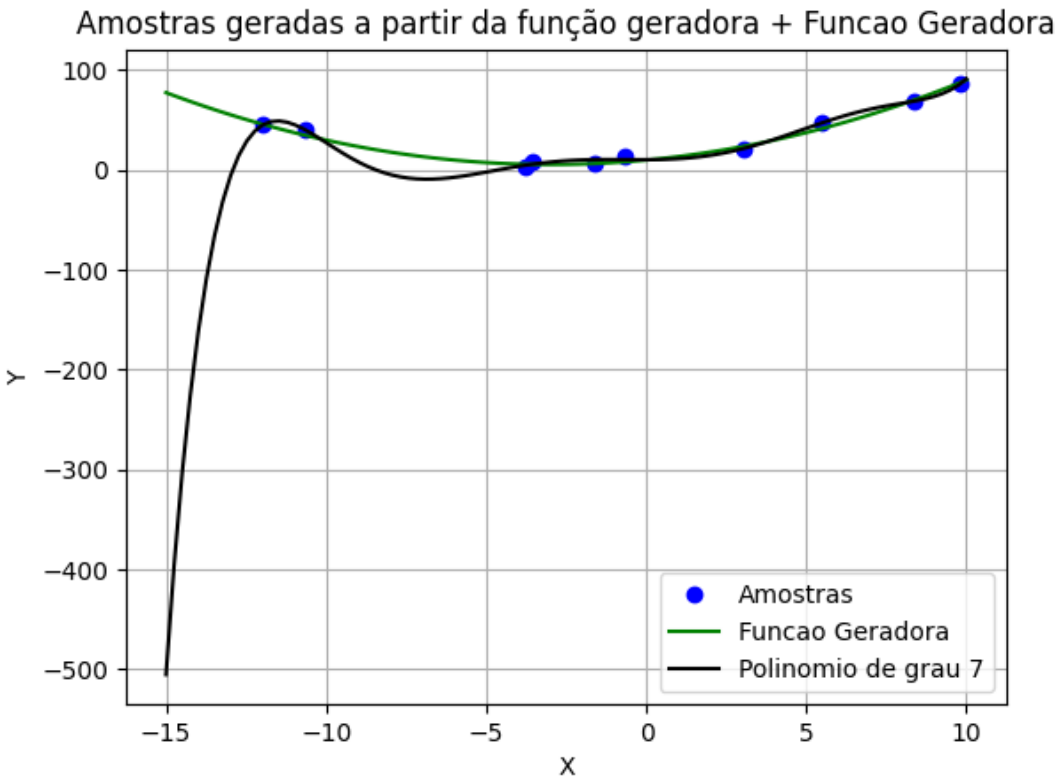

Amostras geradas a partir da função geradora + Funcao Geradora
Amostras
Funcao Geradora
Polinomio de grau 2
Y
X
0
20
40
60
80
−15
−10
−5
0
5
10

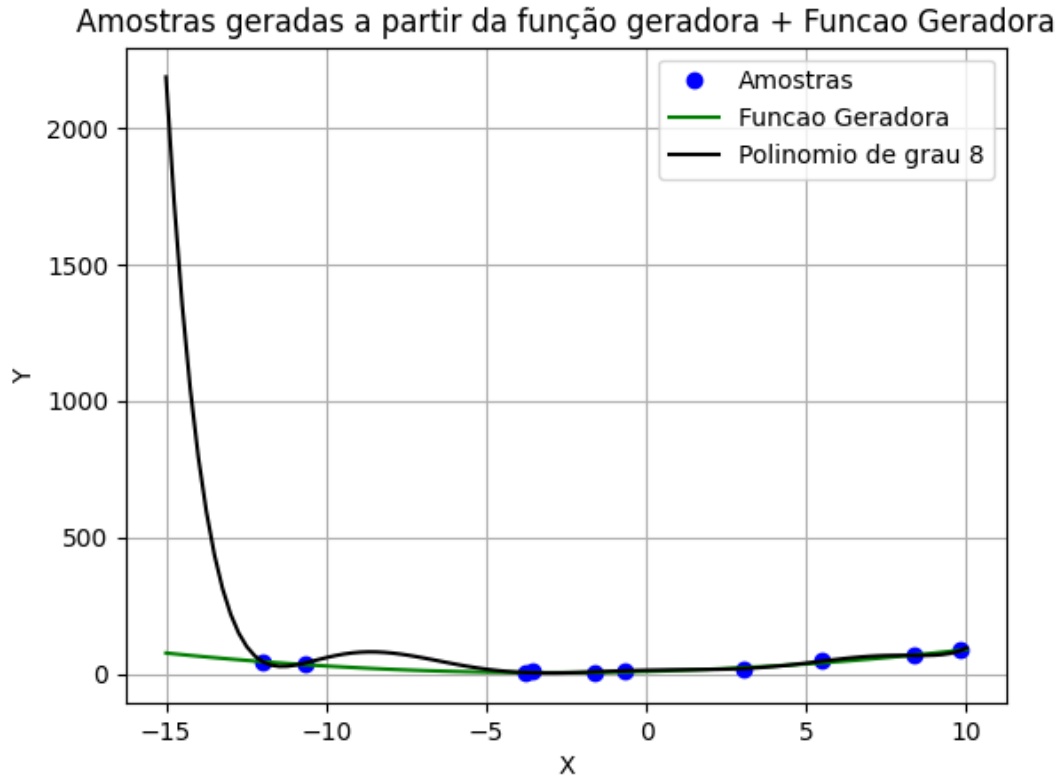

Amostras geradas a partir da função geradora + Funcao Geradora
Amostras
Funcao Geradora
Polinomio de grau 3
Y
X
0
20
40
60
80
−15
−10
−5
0
5
10


Amostras geradas a partir da função geradora + Funcao Geradora
Amostras
Funcao Geradora
Polinomio de grau 4
Y
X
0
20
40
60
80
−15
−10
−5
0
5
10


Amostras geradas a partir da função geradora + Funcao Geradora
Amostras
Funcao Geradora
Polinomio de grau 5
Y
X
80
60
40
20
0
−15
−10
−5
0
5
10


Amostras geradas a partir da função geradora + Funcao Geradora
Amostras
Funcao Geradora
Polinomio de grau 6
Y
X
80
60
40
20
0
−15
−10
−5
0
5
10


Amostras geradas a partir da função geradora + Funcao Geradora
100
0
−100
−200
−300
−400
−500
Y
−15
−10
−5
0
5
10
X
Amostras
Funcao Geradora
Polinomio de grau 7

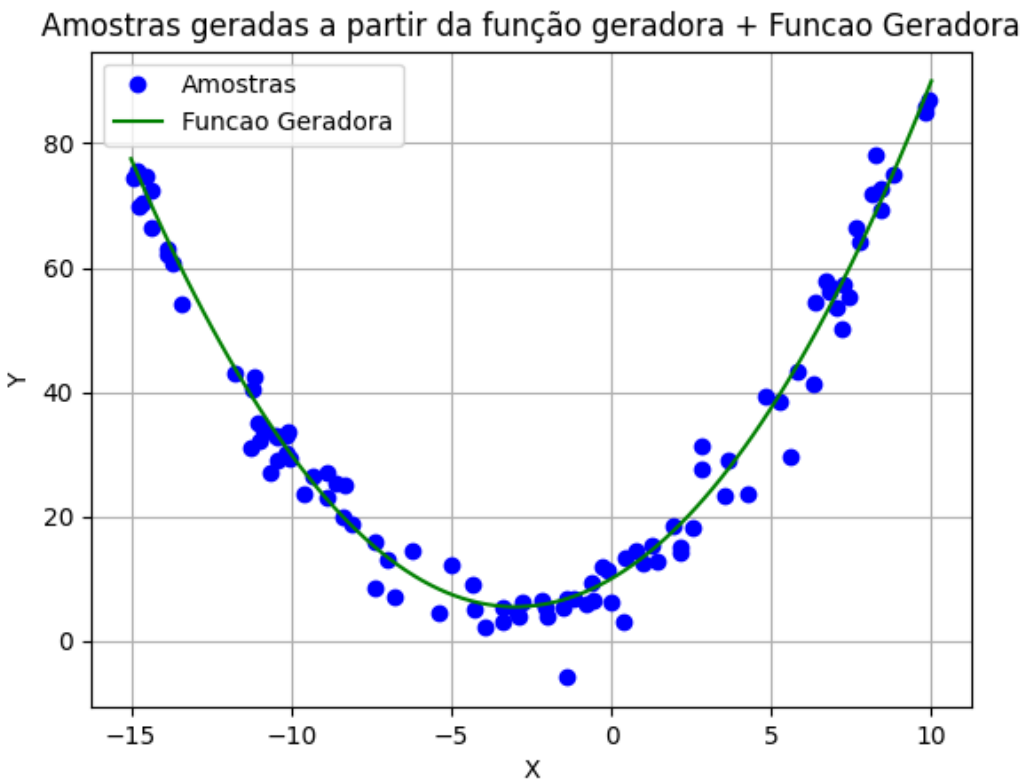

Amostras geradas a partir da função geradora + Funcao Geradora
2000
1500
1000
500
0
Y
−15
−10
−5
0
5
10
X
Amostras
Funcao Geradora
Polinomio de grau 8

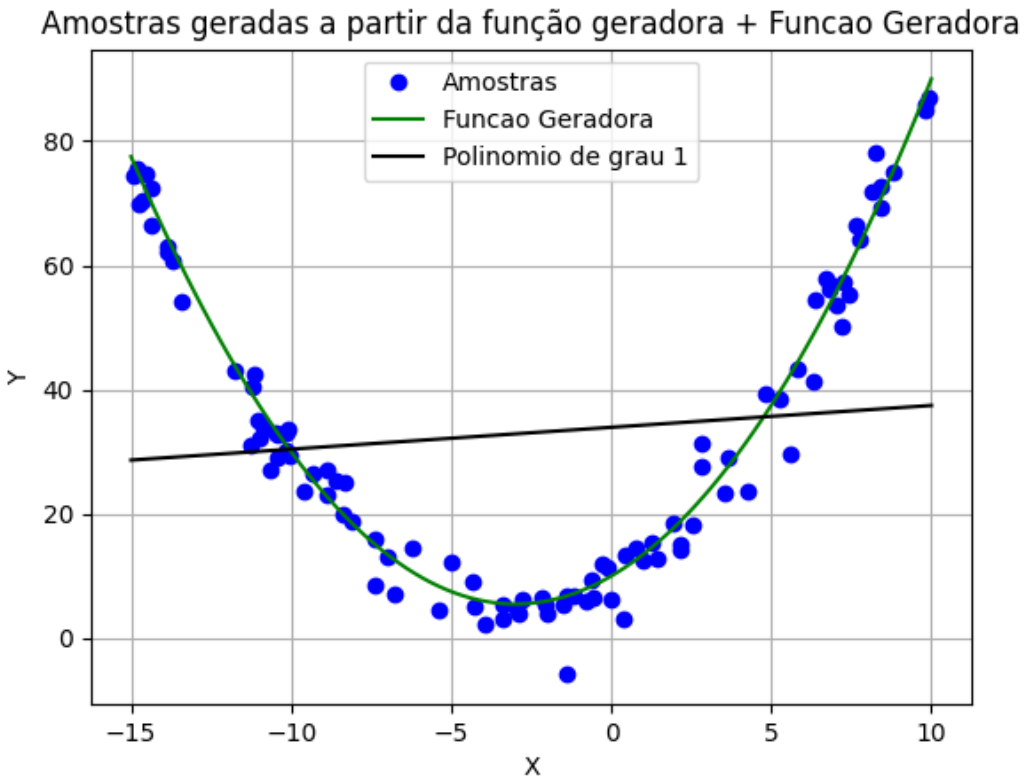
Para 100 pontos

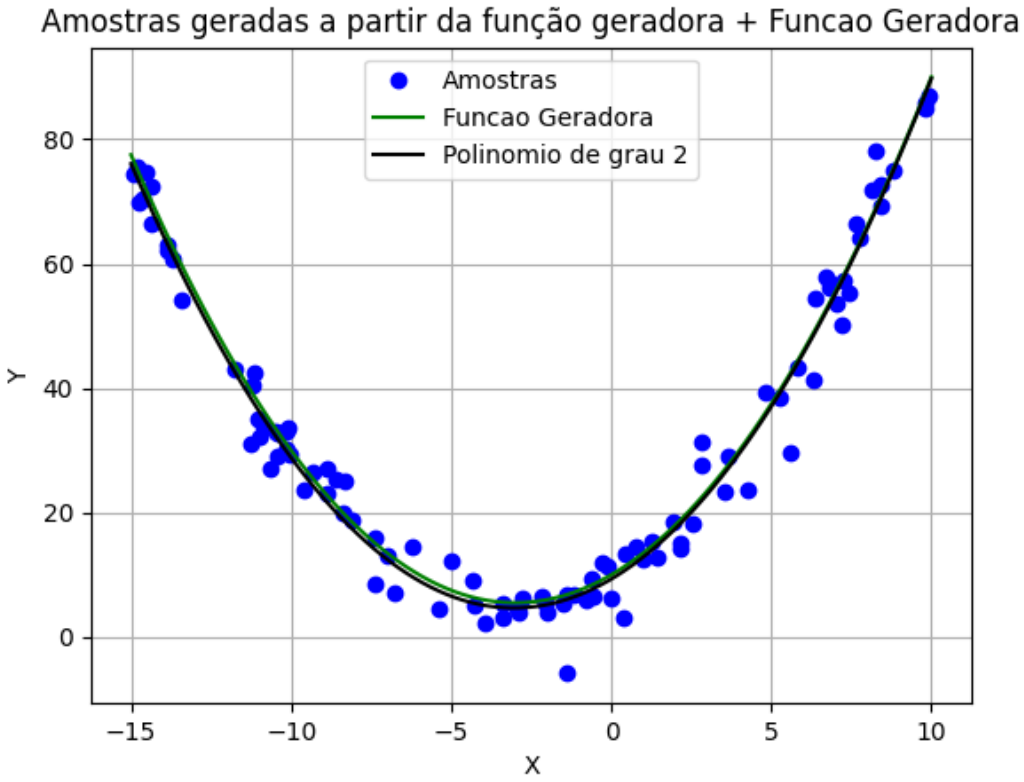
-Polinômios obtidos

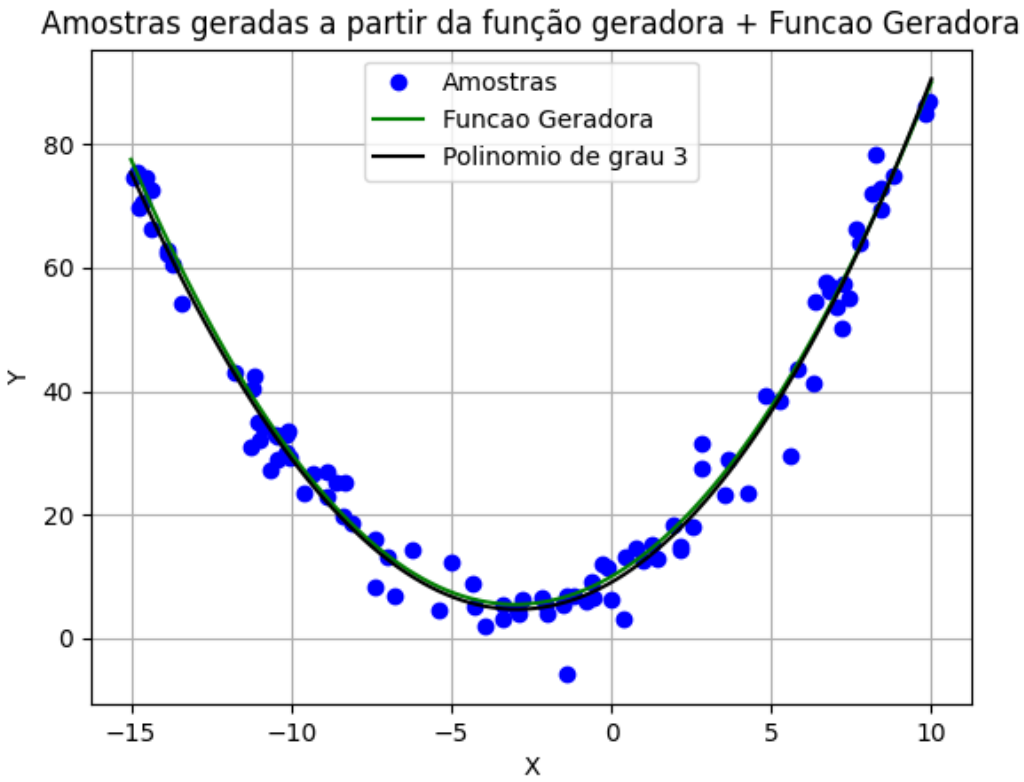
```
Para p = 1: [33.93451137  0.35183893]
Polinomio obtido:
Y = 0.3518X^1 + 33.9345
Para p = 2: [9.30680006 3.04681368 0.49980302]
Polinomio obtido:
Y = 0.4998X^2 + 3.0468*X^1 + 9.3068
Para p = 3: [9.06524338e+00 2.96908902e+00 5.07794827e-01 1.00567722e-03]
Polinomio obtido:
Y = 0.0010X^3 + 0.5078*X^2 + 2.9691*X^1 + 9.0652
Para p = 4: [ 9.03565192e+00  2.98830664e+00  5.10098491e-01  7.06244809e-04
 -2.57919564e-05]
Polinomio obtido:
Y = -0.0000X^4 + 0.0007*X^3 + 0.5101*X^2 + 2.9883*X^1 + 9.0357
Para p = 5: [ 8.54087292e+00  2.80356442e+00  5.61921862e-01  6.46728857e-03
 -5.83066169e-04 -4.59044251e-05]
Polinomio obtido:
Y = -0.0000X^5 + -0.0006*X^4 + 0.0065*X^3 + 0.5619*X^2 + 2.8036*X^1 + 8.5409
Para p = 6: [ 8.51011863e+00  2.86619330e+00  5.69316169e-01  3.97012603e-03
 -7.81403542e-04 -2.61733963e-05  1.42102769e-06]
Polinomio obtido:
Y = 0.0000X^6 + -0.0000*X^5 + -0.0008*X^4 + 0.0040*X^3 + 0.5693*X^2 + 2.8662*X^1 + 8.5101
Para p = 7: [ 9.08241071e+00  2.91024519e+00  4.43768004e-01 -1.56624634e-03
  2.47190744e-03  1.39808199e-04 -1.95296941e-05 -1.23763398e-06]
Polinomio obtido:
Y = -0.0000X^7 + -0.0000*X^6 + 0.0001*X^5 + 0.0025*X^4 + -0.0016*X^3 + 0.4438*X^2 + 2.9102*X^1 + 9.0824
Para p = 8: [ 9.24133559e+00  3.45925104e+00  4.21678142e-01 -4.57327235e-02
  2.14907017e-03  1.00372075e-03  8.73199792e-06 -6.07467014e-06
 -2.39690744e-07]
Polinomio obtido:
Y = -0.0000X^8 + -0.0000*X^7 + 0.0000*X^6 + 0.0010*X^5 + 0.0021*X^4 + -0.0457*X^3 + 0.4217*X^2 + 3.4593*X^1 + 9.2413
```

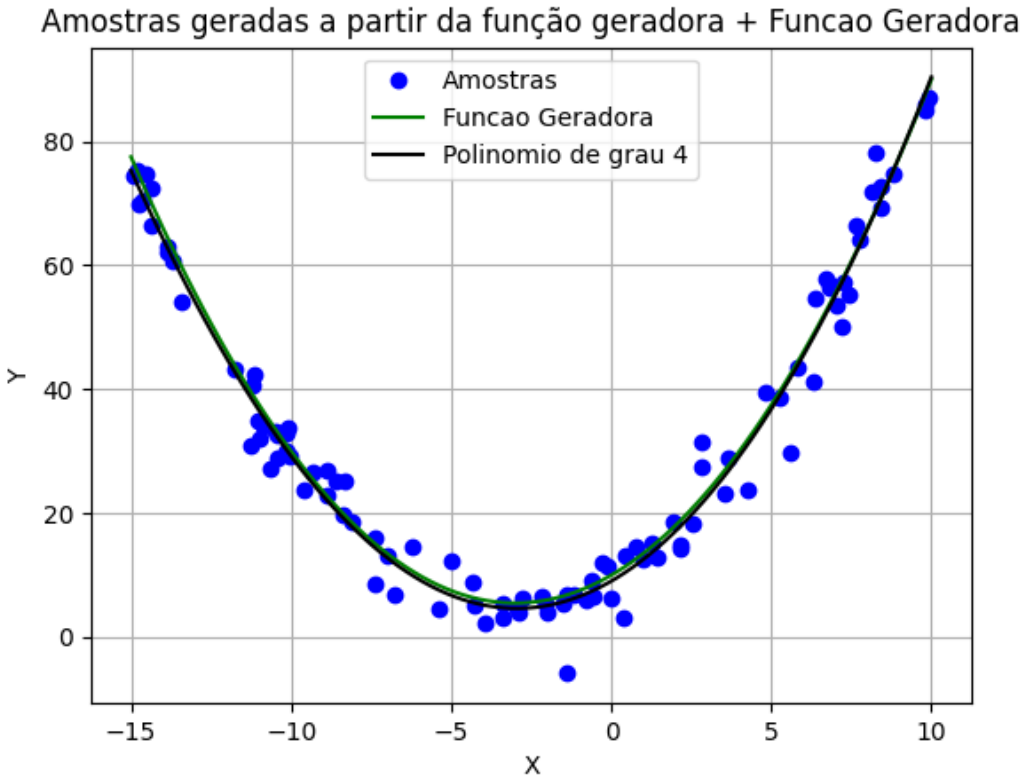
-Gráficos obtidos

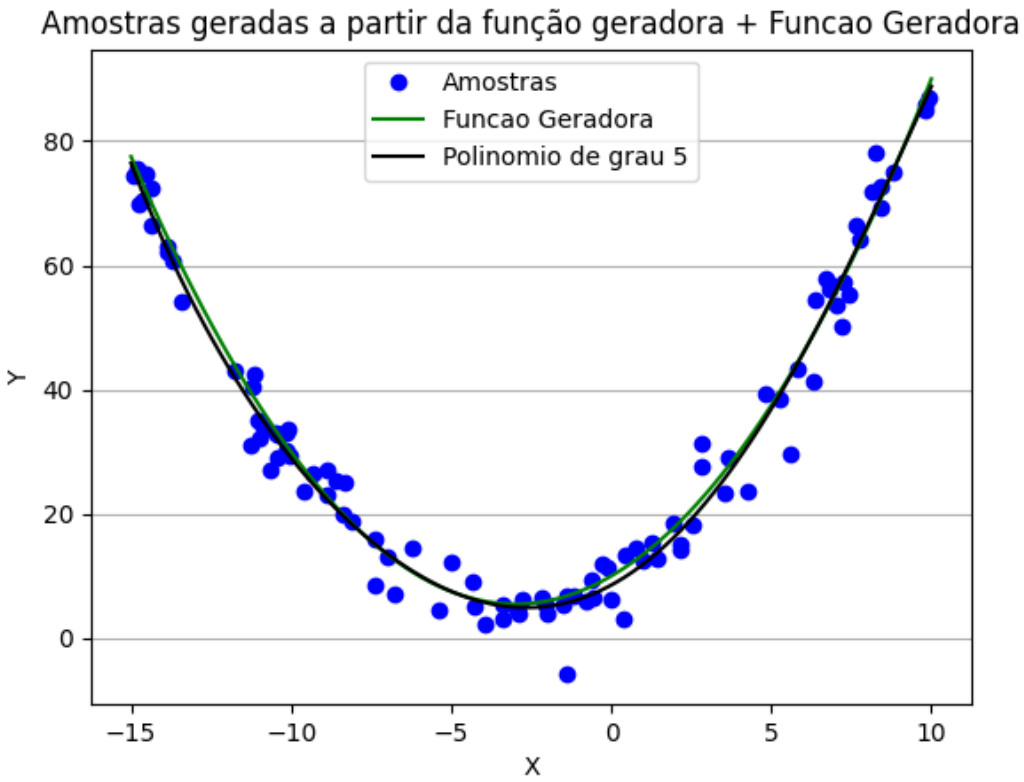


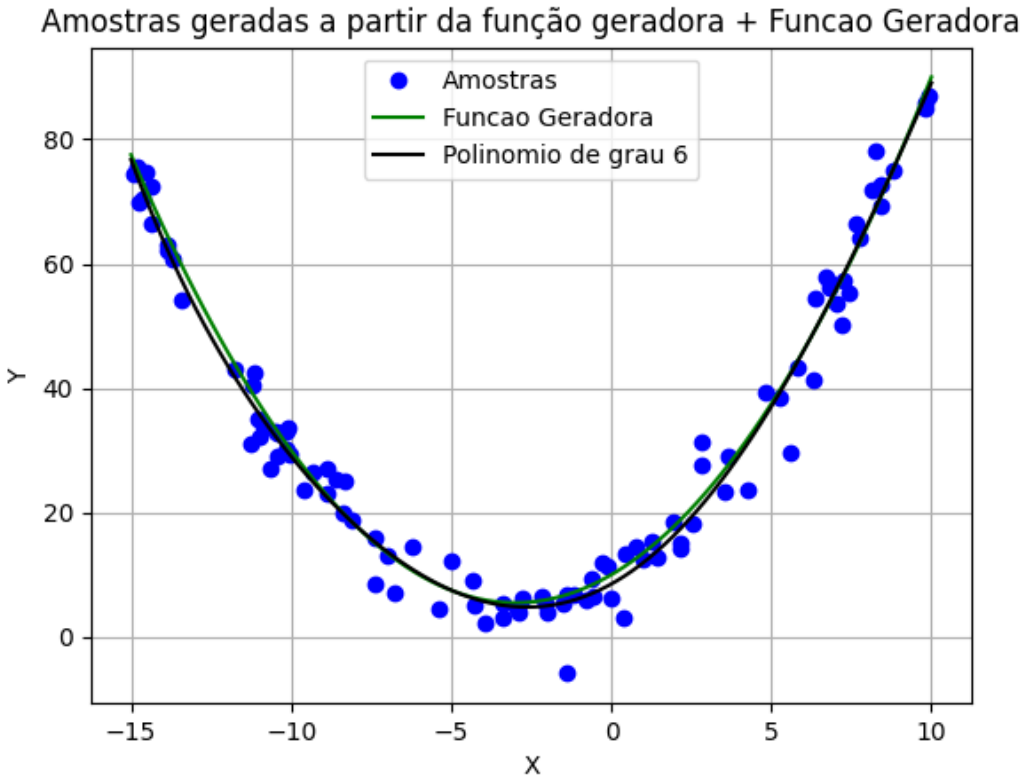

Amostras geradas a partir da função geradora + Funcao Geradora
Amostras
Funcao Geradora
Polinomio de grau 1
Y
X
80
60
40
20
0
−15
−10
−5
0
5
10

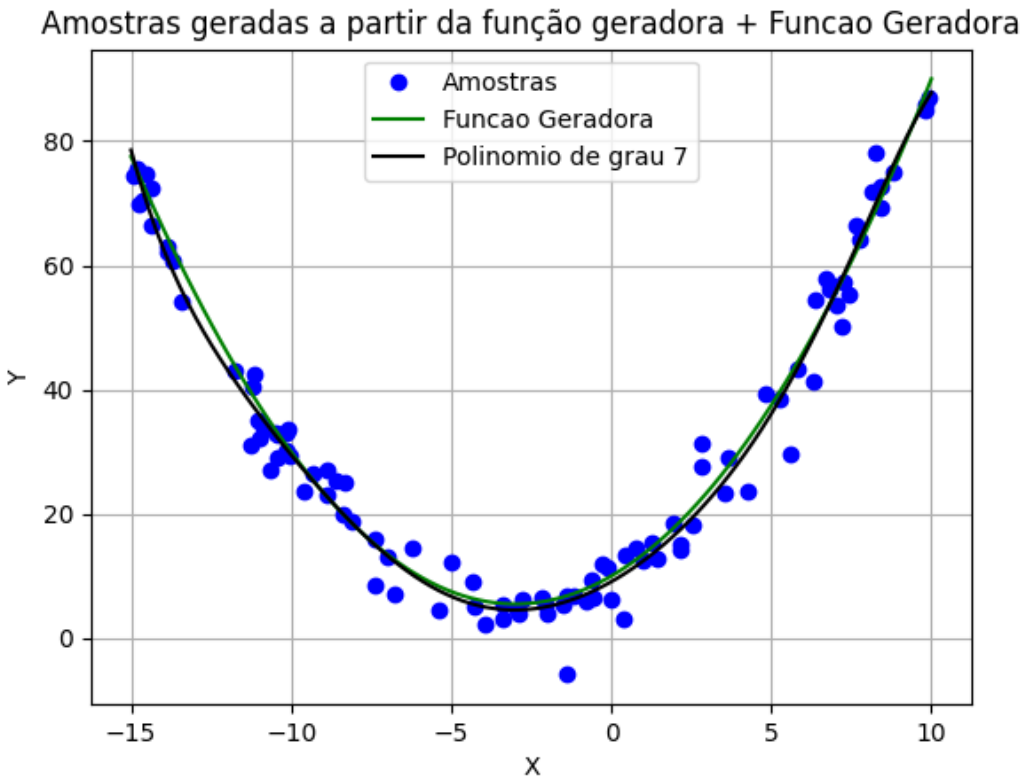

Amostras geradas a partir da função geradora + Funcao Geradora
Amostras
Funcao Geradora
Polinomio de grau 2
Y
X
80
60
40
20
0
−15
−10
−5
0
5
10

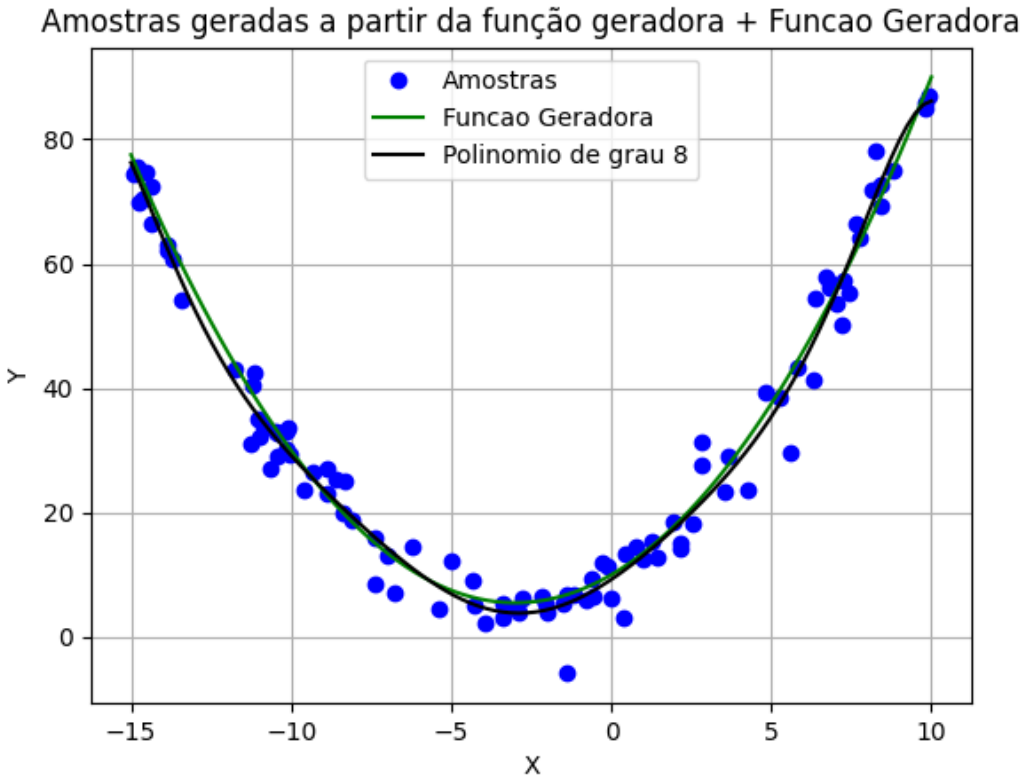

Amostras geradas a partir da função geradora + Funcao Geradora
Amostras
Funcao Geradora
Polinomio de grau 3
Y
X
80
60
40
20
0
−15
−10
−5
0
5
10


Amostras geradas a partir da função geradora + Funcao Geradora
Amostras
Funcao Geradora
Polinomio de grau 4
Y
X
80
60
40
20
0
−15
−10
−5
0
5
10


Amostras geradas a partir da função geradora + Funcao Geradora
Amostras
Funcao Geradora
Polinomio de grau 5
Y
X
80
60
40
20
0
−15
−10
−5
0
5
10


Amostras geradas a partir da função geradora + Funcao Geradora
Amostras
Funcao Geradora
Polinomio de grau 6
Y
X
80
60
40
20
0
−15
−10
−5
0
5
10


Amostras geradas a partir da função geradora + Funcao Geradora
Amostras
Funcao Geradora
Polinomio de grau 7
Y
X
80
60
40
20
0
−15
−10
−5
0
5
10


Amostras geradas a partir da função geradora + Funcao Geradora
Amostras
Funcao Geradora
Polinomio de grau 8
Y
X
80
60
40
20
0
−15
−10
−5
0
5
10

- O underfitting foi observado na resolução com o polinômio de primeira ordem, tanto para 10 pontos quanto para 100 pontos.
- O overfitting foi oibservado apenas para as resoluções com polinômios de ordem 5,6,7 e 8 para 10 pontos.